Application Based Questions – Answers

Course: Cloud Computing and Big Data Analytics (CSA15)
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments (BL3)
Total Marks: 25 (5 questions)

Question 1: Auto-Scaling and Load Balancing for Seasonal Traffic Spikes

Scenario: An online retailer faces unpredictable traffic spikes during seasonal sales.

How auto-scaling and load balancing handle demand

- **Auto-scaling** automatically adds or removes virtual machine or container instances based on rules (for example, CPU above 70%, request count per instance, or a schedule). *Horizontal scaling* (more instances) is preferred over *vertical scaling* (bigger instances) because it has no upper limit and causes no restart.
- **Load balancing** distributes incoming requests across all healthy instances using algorithms such as round robin, least connections or weighted routing. It runs health checks and stops sending traffic to failed instances.
- Together, auto-scaling supplies capacity and the load balancer spreads the work across it, so no single server becomes a bottleneck. When the sale ends, extra instances are terminated, so the company pays only for what it used.

Reducing downtime and improving user experience

- Failed or overloaded servers are replaced automatically, and instances are spread across multiple Availability Zones, so one failure does not take the site down.
- Response times stay consistent during peaks, so pages load quickly and checkout does not time out. This reduces cart abandonment and protects revenue and brand reputation.
- Rolling updates can be done behind the load balancer without taking the site offline.

Role of monitoring tools

- Tools such as Amazon CloudWatch, Azure Monitor or Prometheus/Grafana collect metrics (CPU, memory, latency, error rate, request count) and logs.
- Metrics trigger scaling policies, alarms notify the operations team, and dashboards show trends. Historical data helps in capacity planning and in setting scheduled scaling before known events.

Real-world example: An e-commerce site such as Amazon or Flipkart during Black Friday or Big Billion Days runs web servers in an Auto Scaling Group behind an Elastic Load Balancer. Traffic jumps tenfold in minutes, CloudWatch detects high CPU and request counts, new instances launch and register with the load balancer, and customers continue to browse and pay without slowdown. After the sale, the group scales back in.

Question 2: Secure and Cost-Effective Cloud Backup Storage

Durability and redundancy

- Object storage services (Amazon S3, Azure Blob Storage, Google Cloud Storage) automatically store multiple copies of data across several devices and facilities, giving very high durability (S3 advertises 99.999999999%, or "eleven nines").
- **Replication** options include multi-AZ, cross-region and geo-redundant storage. **Erasure coding**, checksums and automatic repair of corrupted copies protect against silent data loss.
- **Versioning** and **object lock (WORM)** protect against accidental deletion and ransomware.

Encryption and access control

- **Encryption in transit** uses TLS/HTTPS. **Encryption at rest** uses AES-256 with provider-managed keys or customer-managed keys in a Key Management Service (KMS/HSM), with key rotation.
- **Access control** uses Identity and Access Management (IAM) with the principle of least privilege, role-based access, bucket policies, multi-factor authentication, blocking of public access, and private network endpoints. Access logs and audit trails record who accessed what.

Cost advantages over traditional storage

- No upfront capital expenditure on hardware, data-centre space, power, cooling or maintenance staff; it is pay-as-you-go (operating expenditure).
- Capacity is elastic, so there is no over-provisioning for future growth.
- **Storage tiers** (hot, cool, archive, such as S3 Glacier) and lifecycle policies move older backups to much cheaper storage automatically.
- Built-in redundancy removes the cost of a second site and tape handling.

Disaster recovery use case: A hospital or bank backs up its databases nightly to cloud storage replicated to a second region. If the primary data centre is destroyed by fire or flood, or ransomware encrypts local files, the organization restores data from the latest clean version in the other region and brings services up on cloud virtual machines. This meets a defined **RPO** (acceptable data loss) and **RTO** (acceptable recovery time) without maintaining a costly secondary data centre.

Question 3: Global Deployment with CDNs and Edge Servers

How CDNs improve performance and reduce latency

- A Content Delivery Network is a geographically distributed network of servers that caches content close to users. Instead of every request travelling to the origin server, it is served from a nearby location, which shortens the round-trip time and reduces the number of network hops.
- Caching static content (images, scripts, video segments, app updates) lowers load on the origin and saves bandwidth. Compression, HTTP/2 or HTTP/3, TCP optimization and TLS termination at the edge further speed up delivery.
- CDNs also absorb traffic spikes and help mitigate DDoS attacks.

How edge servers enhance user experience

- Edge servers (points of presence) in each region serve users from the nearest location, so a user in Mumbai and a user in London both get low latency.
- Edge computing (for example Lambda@Edge or Cloudflare Workers) can run authentication, personalization, A/B testing and API routing at the edge, so even dynamic requests do not always travel to the origin.
- Intelligent routing sends users to the nearest healthy edge and reroutes around congestion or failures.

How cloud providers support global availability

- Providers operate **regions and Availability Zones** worldwide. The app can be deployed in several regions with global load balancing (for example Route 53 latency-based or geo-routing, Azure Front Door), cross-region data replication and automatic failover.
- This gives resilience to regional outages, compliance with local data rules, and consistent performance for all users.

Real-time streaming/gaming example: A video streaming platform such as Netflix stores popular titles on edge servers inside or near ISPs, so playback starts quickly and does not buffer. An online multiplayer game uses regional game servers and edge routing to keep ping low, while the CDN delivers patches and game assets quickly. Lower latency means smoother video, fewer lag spikes and higher user retention.

Question 4: Compliance, Security and Data Sovereignty for a Financial Institution

Regulatory compliance and auditing

- Major providers hold independent certifications and attestations, such as ISO 27001, SOC 1/2/3, PCI DSS, and support for GDPR, HIPAA and regional financial rules (for example RBI guidelines in India). Compliance reports are available to customers on demand (AWS Artifact, Azure Compliance Manager).
- The **shared responsibility model** applies: the provider secures the underlying infrastructure, while the institution secures its data, identities, configuration and applications.
- Auditing is supported by policy tools (AWS Config, Azure Policy), immutable audit trails and the right to audit through third-party assessments.

Role of logging, monitoring and encryption

- **Logging** (AWS CloudTrail, Azure Activity Log) records every API call and user action, giving a tamper-evident audit trail for investigations and regulators.
- **Monitoring** and SIEM tools (CloudWatch, Azure Sentinel, GuardDuty) detect anomalies, unauthorized access and fraud patterns in real time and raise alerts.
- **Encryption** protects confidentiality: TLS in transit, AES-256 at rest, field-level encryption for card and account numbers, and keys held in an HSM under the institution's control (BYOK/HYOK). Combined with IAM, MFA and network segmentation, this limits the impact of a breach.

Ensuring data sovereignty

- Choose cloud regions inside the required country or jurisdiction and use policies (for example service control policies or Azure Policy) that deny creation of resources elsewhere.

- Disable or restrict cross-region replication and backups, and keep encryption keys in the same jurisdiction.
- Use provider sovereign-cloud offerings where available, include data-residency clauses in contracts, and verify through audits and data-location reports.

Example: A bank processes online payments and UPI/card transactions. Card data is tokenized and encrypted, the workload runs only in a domestic region, every access to transaction records is logged by CloudTrail, and anomaly detection flags unusual transfers for fraud review. During a regulatory audit, the bank presents the logs, the provider's PCI DSS certificate and proof that data never left the country.

Question 5: Containers, Orchestration and Portability

Containers versus virtual machines

Aspect	Virtual Machine	Container
Virtualization level	Hardware, via a hypervisor	Operating system, shares the host kernel
Contents	Full guest OS plus app	App plus libraries only
Size	Gigabytes	Megabytes
Startup time	Minutes	Seconds or milliseconds
Resource usage	High (each VM needs its own OS, CPU, RAM)	Low; more workloads per host
Isolation	Strong (separate kernels)	Lighter (shared kernel), so a larger attack surface
Performance	Some hypervisor overhead	Near-native

Containers (for example Docker) therefore give higher density and faster deployment, while VMs give stronger isolation and can run different operating systems.

How orchestration (Kubernetes) simplifies deployment and scaling

- Kubernetes schedules containers (in pods) across a cluster of nodes and automates deployment, rollouts and rollbacks from declarative configuration files.
- It provides **self-healing** (restarts or reschedules failed containers), **horizontal auto-scaling** based on load, **service discovery and load balancing**, secrets and configuration management, and rolling updates with zero downtime.
- This removes manual work and makes large numbers of containers manageable and consistent.

Portability across cloud environments

- A container image packages the application with all dependencies, so it runs the same way on a developer laptop, on-premises servers, AWS, Azure or Google Cloud ("build once, run anywhere").

- Kubernetes is available as a managed service on all major clouds (EKS, AKS, GKE) and on-premises, which supports hybrid and multi-cloud strategies, avoids vendor lock-in and eases migration.

Example of a microservices-based application: An online food-delivery app is split into independent services: user/authentication, restaurant catalogue, order, payment, delivery tracking and notification. Each runs in its own container, can be written in a different language and be scaled independently (the order service scales up at dinner time). Kubernetes manages them, an API gateway routes requests, and a failure in the notification service does not bring down ordering or payment. Teams can update one service and deploy it without redeploying the entire application.

End of answers.